

## **Experiment-10**

### **Program to Implement A\* Algorithm**

#### **Aim**

To develop a Python program to implement the A\* Search Algorithm for finding the shortest path.

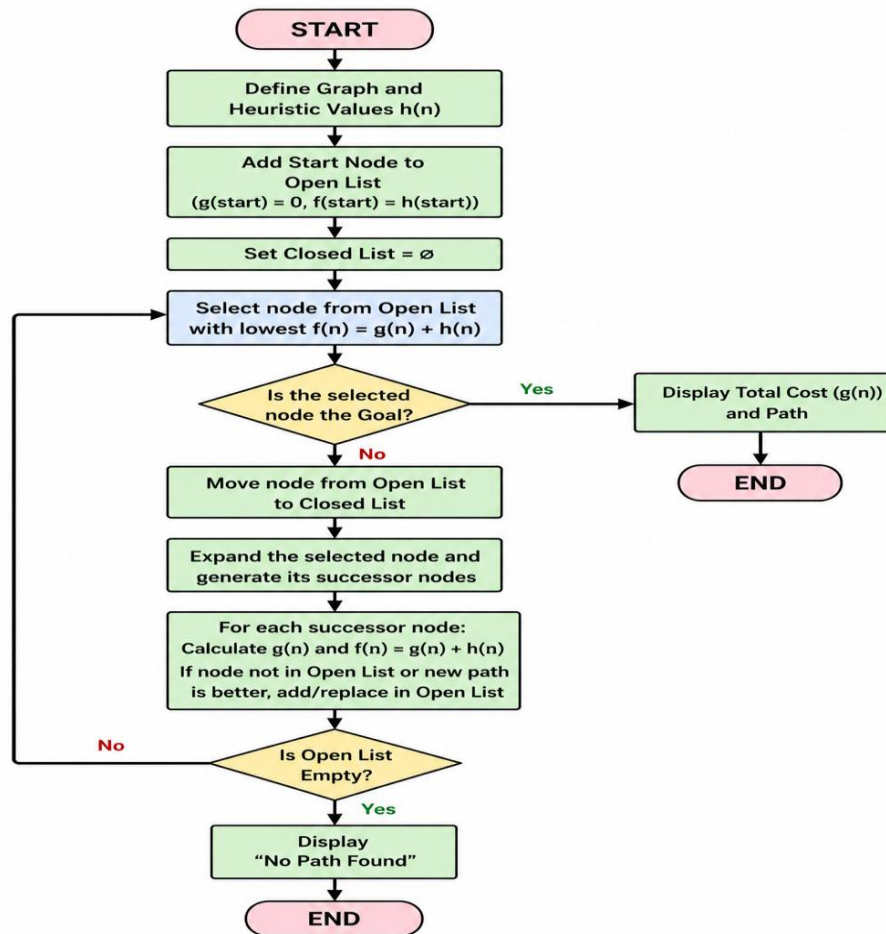
#### **Objective**

- To implement the A\* Search Algorithm.
- To find the shortest path between the start and goal nodes.
- To understand heuristic-based search techniques in Artificial Intelligence.

#### **Algorithm**

1. Start the program.
2. Define the graph and heuristic values.
3. Add the start node to the open list.
4. Select the node with the lowest  $f(n) = g(n) + h(n)$  value.
5. If the goal node is reached, display the cost.
6. Otherwise, expand the selected node.
7. Update the open list with neighboring nodes.
8. Repeat until the goal is reached.
9. Stop the program.

### Flowchart:



### Code:

```
graph = {
    'A': {'B': 1, 'C': 3},
    'B': {'D': 5, 'E': 2},
    'C': {'F': 4},
    'D': {},
    'E': {'G': 1},
    'F': {'G': 2},
    'G': {}
}
```

```
h = {'A':7, 'B':6, 'C':4, 'D':5, 'E':2, 'F':2, 'G':0}
```

```
open = [('A', 0)]
```

```
visited = {}
```

```
while open:
```

```
    open.sort(key=lambda x: x[1] + h[x[0]])
```

```
    node, cost = open.pop(0)
```

```
    if node == 'G':
```

```
        print("Cost =", cost)
```

```
        break
```

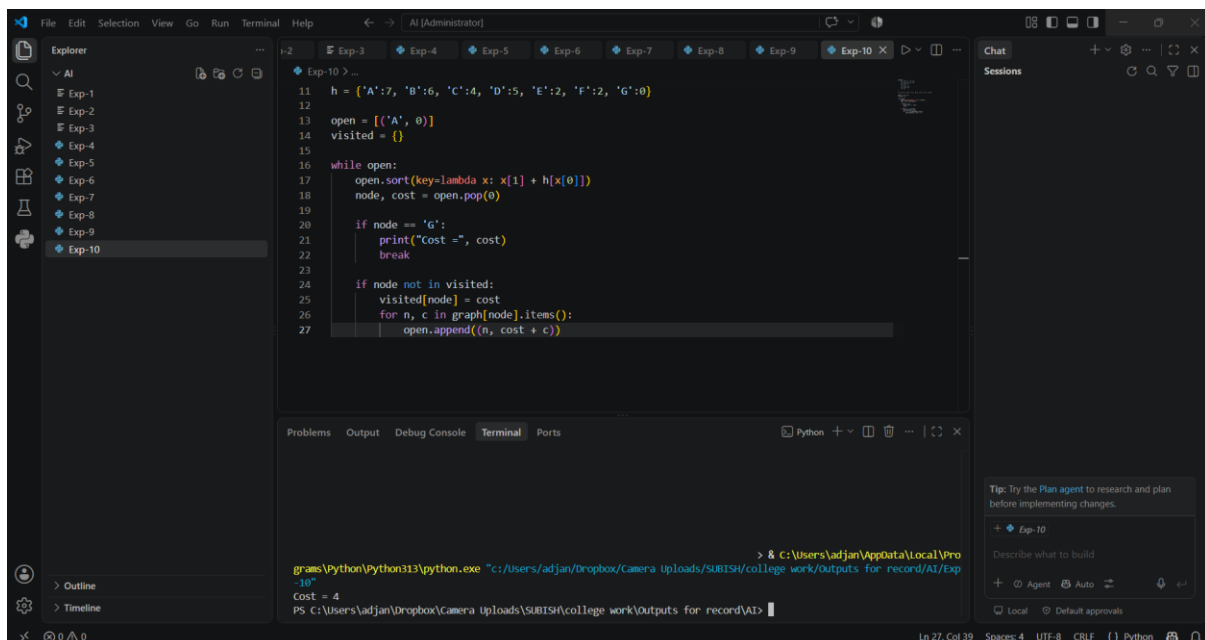
```
    if node not in visited:
```

```
        visited[node] = cost
```

```
        for n, c in graph[node].items():
```

```
            open.append((n, cost + c))
```

## Output:



The screenshot shows a VS Code editor with a Python script in the main editor and its output in the terminal. The script is a Dijkstra's algorithm implementation for finding the shortest path from node 'A' to node 'G'. The graph is defined as follows:

```
graph = {'A': {'B': 6, 'C': 4, 'D': 5, 'E': 2, 'F': 2, 'G': 0},
        'B': {'A': 6, 'C': 4, 'D': 5, 'E': 2, 'F': 2, 'G': 0},
        'C': {'A': 4, 'B': 6, 'D': 5, 'E': 2, 'F': 2, 'G': 0},
        'D': {'A': 5, 'B': 6, 'C': 4, 'E': 2, 'F': 2, 'G': 0},
        'E': {'A': 2, 'B': 6, 'C': 4, 'D': 5, 'F': 2, 'G': 0},
        'F': {'A': 2, 'B': 6, 'C': 4, 'D': 5, 'E': 2, 'G': 0},
        'G': {'A': 0, 'B': 6, 'C': 4, 'D': 5, 'E': 2, 'F': 2}}
```

The script defines a priority queue 'open' and a visited set. It sorts the queue by the sum of the current cost and the heuristic cost 'h'. The output in the terminal shows the cost of the shortest path from 'A' to 'G' is 4.

```
> & C:\Users\adjan\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/adjan/Dropbox/Camera Uploads/SUBISH/college work/Outputs for record/AI/Exp-10"
Cost = 4
PS c:\Users\adjan\Dropbox\Camera Uploads\SUBISH\college work\Outputs for record\AI>
```

**Result:**

The A\* algorithm was successfully implemented and found the shortest path to the goal node.

**Conclusion:**

The A\* algorithm efficiently finds the shortest path by combining the actual path cost and heuristic estimate, making it one of the most effective informed search algorithms in Artificial Intelligence.